Lecture 18

Understanding SAM/BAM files.

The trials and tribulations of a single read pair

Imagine a situation where a `150 bp` DNA fragment of the Ebola genome gets sequenced in **paired-end** mode.

```
              150
               |
==================================================
```

Two reads will be generated.

These are their stories.



Fragmentation



Select one of the strands (forward or reverse)



Produce an even shorter sequence from each end



But we don't know the orientation yet

While we indicated the reads as

```
---->
        <----
```

We have not aligned them yet, so all we have are

```
----
----
```

One goes in the first file: **read 1** (first in pair)

The other in the second file: **read 2** (second in pair)

Let's get the actual data

```
wget http://data.biostarhandbook.com/data/pairs.tar.gz
tar zxvf pairs.tar.gz
```

The files contain:

```
@read1
ACACTGCTCACCAACAGCACCACAGGAAATCCAGAACACACAAGTCAAGA
+
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
```

and (note how the read names are identical)

```
@read1
TATGTTGGATGGAGATGGAGGTTGTGATAGGTATTCGGATGTTGTATAGA
+
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
```

Make a BAM file

```
# Get and index the genome
REF=db/ebola.fa
mkdir -p db
efetch -db nuccore -id AF086833 -format fasta > $REF
bwa index $REF
```

A `BAM` file is a sorted and indexed `SAM` file:

```
bwa mem $REF pair1.fq pair2.fq | samtools sort > bwa.bam
```

Index the bam file:

```
samtools index bwa.bam
```

Viewing the BAM file

```
samtools view bwa.bam
```

Two lines, first in pair:

```
read1   97      AF086833.2      1       60      50M     =
101     150     CGGACACACAAAAAGAAAGAAGAATTTTTAGGATCTTTTGTG
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
NM:i:0  MD:Z:50 MC:Z:50M        AS:i:50 XS:i:0
```

and second in pair has:

```
read1   145     AF086833.2      101     60      50M     =
1       -150    TTAAATTGAAATTGTTACTGTAATCACACCTGGTTTGTTTCA
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
NM:i:0  MD:Z:50 MC:Z:50M        AS:i:50 XS:i:0
```

What are the positions FLAG and POS

```
samtools view bwa.bam | cut -f 2, 4
```

prints:

```
97      1
145     101
```

investigate flags:

```
samtools flags 97
0x61    97      PAIRED,MREVERSE,READ1

samtools flags 145
0x91    145     PAIRED,REVERSE,READ2
```

What we know so far

- `READ1` aligns at position 1
- `READ2` aligns at postion 101 on the reverse strand

Note how the FLAG and many other fields in the SAM format try to give you information on the "mate".

This is an excellent feature. We don't have to look up the mate to get information on it.

The MAPQ (mapping quality) and CIGAR

```
samtools view bwa.bam | cut -f 5,6
```

Shows

```
60      50M
60      50M
```

Now as mentioned before MAPQ are estimates at best. The probability of making: in this case: 1/10^6 (6 zeros)

CIGAR: `50M` (50 matches or mismatches)

Why is the CIGAR string so useless?

It is trying to capture the lenght of the semi-global alignment.

A a match or mismatch does not change the length of the alignment in neither the reference nor the query.

`M` = Match or Mismatch

Bad idea or a big mistake?

How can I tell if there is an actual mismatch? Look at the optional `MD` tag (see later)

How can you tell where the alignment ends in the reference?

`POS` + `CIGAR` operations

If CIGAR is `50M`

```
end1 = 1 and 50M
end2 = 101 and 50M
```

It is not a sum because the `50M` the start. So it is `1 + 50 - 1` and `101 + 50 - 1`

So the first read aligns between `[1, 50]`, the second read aligns `[101, 150]`

It gets better (not!)

Some `CIGAR` strings modify the alignment lenght others do not:

`D` moves the alignment end, but `I` does not since it does not exist in the reference.

If `POS=1` for `CIGAR` of 10 match/mismatches, 2 deletions, 10M, 5 insertions then 10M

```
10M2D10M5I10M

1 + 10 + 2 + 10 + 10 - 1
```

So the alignment spans `[1, 32]`

More on alignment lenghts

Because it is so difficult to get this right we use tools to compute them.

But the alignment length is needed very frequently.

Shouldn't essential information like alignment lenght be in the SAM format to begin with?

And now you know why I vent so much against the SAM format. 😈

🙏 Mapping Gods please design a better format!

Mate related information: RNEXT, PNEXT, TLEN

```
samtools view bwa.bam | cut -f 7,8,9
```

RNEXT tells us where the other read aligns. If it is `=` it means the same segment (chromosome).

```
=       101     150
=       1       -150
```

`PNEXT` is the `POS` fiels of the mate. So you don't have to look it up yourself.

`TLEN` is the observed fragment length. Leftmost position to rightmost position in the reference.

The TLEN field

The TLEN tries to capture the apparent lenght of the fragment based on the alignment in the reference.

It it the cumulative alignment lenght of both pairs.

```
      |<--    TLEN    -->|
==============================
      ---->
                    <----
```

If SAM gave you the alignment lenght of each read it would be trivial to compute it.

But since it does more simple and error prone computation is required.

Trivial but extremely error prone computations

To understand the `TLEN` you need the `POS` of both and the `CIGAR` of the rightmost alignment. Pfff...

```
samtools view bwa.bam | cut -f 4,6,8,9
1       50M     101     150
101     50M     1       -150
```

The leftmost `POS` is `1`. The righmost `POS` is `101`.

The end of the alignment is 101 + 50 - 1 = 150

The **distance** from 1 to the 150 -> 150
For the mate the distance from 150 to 1 -> -150

Why is such a simple thing so complicated?

It will be your job to change it!

The SEQ and QUAL columns.

```
samtools view bwa.bam | cut -f 10
CGGACACACAAAAAGAAAGAAGAATTTTTAGGATCTTTTGTGTGCGAATA
TTAAATTGAAATTGTTACTGTAATCACACCTGGTTTGTTTCAGAGCCACA
```

Firs read matches:

```
$ cat pair1.fq | head -2
@read1
CGGACACACAAAAAGAAAGAAGAATTTTTAGGATCTTTTGTGTGCGAATA
```

But for the second read is the reverse complement:

```
cat pair2.fq | head -2
@read1
TGTGGCTCTGAAACAAACCAGGTGTGATTACAGTAACAATTTCAATTTAA
```

The rule of SAM

All information is specified relative to the forward strand.

This helps avoid ambiguities with strands.

But remember that it alters the data.

Reads that align to the reverse strand will be shown transformed to the forward strand!

You can covert them back to original orientiation:

```
samtools fastq bwa.bam
```

Optional fields

These are those beyond the 11 mandatory fields.

This is where substantial additional information is encoded.

```
samtools view bwa.bam | cut -f 12-15
```

Each tag is form `TAG:TYPE:VALUE`. But the columns are unordered.

```
NM:i:0  MD:Z:50 MC:Z:50M        AS:i:50
NM:i:0  MD:Z:50 MC:Z:50M        AS:i:50
```

Edit one of the pairs and change a base

We replaced one `A` with a `T`

```
bwa mem $REF pair1.fq pair2.fq | samtools sort > mut.bam
```

Look at CIGAR string as well:

```
samtools view mut.bam | cut -f 6,12-15
50M     NM:i:1    MD:Z:11A38   MC:Z:50M     AS:i:45
50M     NM:i:0    MD:Z:50      MC:Z:50M     AS:i:50
```

The CIGAR string still shows `50M` but the `MD` tag shows 11 matches, a mismatch to an `A` in the reference then 38 matches.

What is the format of MD tags?

It is another `CIGAR` like format but not the same.

For example a `50M` in the first format will look like `11A38` in the second. Note how `11 + 1 + 38 = 50`.

Does this mean that the SAM format supports two different kinds of Compact Idiosyncratic Gapped Alignment Reports?

Yes.

🙏 Mapping Gods please design a better format!

What do NM, MX, AS mean?

Refer to the SAM Tags document:

- `NM` number of mismatches
- `AS` alignment score
- `MC` CIGAR string for mate/next segment

Tags are all optional and may only be present when that line (alignment) has certain properties.